

# Projet Cybersecurite-AP1 — Mise en place d'un système de sauvegarde automatisée et sécurisée d'une base de données

Projet du 12/11/2025



l'école d'ingénierie  
informatique



l'école [ tech ]  
de l'expertise digitale

## Participants :

Kaélian BAUDELET

## Projet encadré par :

Julien LE GALES

## Atelier Cybersécurité — Deuxième Année, 2025

EPSI — Ecole de l'ingénierie  
informatique, Arras

## CONSIGNE DU PROJET

### AP – Projet 1 - Mise en place d'un système de sauvegarde automatisée et sécurisée d'une base de données

Vous devez concevoir et développer un système permettant d'assurer la sauvegarde automatisée et la restauration automatisées d'une base de données MariaDB. Ce projet est à réaliser en équipe en utilisant la méthode SCRUM.

Le système devra respecter les étapes suivantes :

- **Exporter la base de données** : Créer un dump de la base de données MariaDB de manière automatisée.
- **Chiffrement** : Chiffrer le fichier de sauvegarde pour garantir la confidentialité des données en cas de transfert.
- **Transfert sécurisé** : Envoyer automatiquement le fichier de sauvegarde chiffré sur un serveur distant.
- **Vérification de l'intégrité** : Mettre en place un mécanisme permettant de vérifier que le fichier de sauvegarde reçu est identique à celui envoyé.
- **Automatisation** : Le processus complet doit être automatisé pour s'exécuter à intervalle régulier sans intervention humaine.
- **Restauration de la base de données** : Développer un système permettant de restaurer la base de données à partir du fichier de sauvegarde chiffré, une fois transféré sur le serveur distant.

#### Contraintes :

- ✓ Vous êtes **libres d'utiliser les technologies**, outils et langages **que vous jugez adaptés** pour réaliser cet exercice (scripts bash, PHP, Python, outils de sauvegarde, protocoles de transfert, etc.).
- ✓ Le système **doit fonctionner de manière autonome** une fois mis en place, sans nécessiter d'intervention manuelle.
- ✓ Le **chiffrement doit garantir la sécurité des données** durant le transfert.
- ✓ Le **transfert du fichier doit se faire de manière sécurisée**, par exemple via un protocole sécurisé (vous devrez justifier votre choix).
- ✓ Un **mécanisme doit être mis en place** pour assurer que le **fichier reçu** sur le serveur distant est **conforme au fichier envoyé** (vérification de l'intégrité).
- ✓ La procédure de restauration doit permettre de récupérer la base de données à son état d'origine.

**Livrables attendus :**

- **Code :** Le code ou script permettant d'automatiser la sauvegarde, le chiffrement, le transfert et la vérification d'intégrité.
- **Documentation :** Un fichier décrivant les choix techniques, les étapes d'installation et d'utilisation du système.
- **Présentation :** Un rapport expliquant votre démarche, les technologies choisies, et comment le système répond aux besoins exprimés.

**Critères d'évaluation :**

*La note pourra varier en fonction de la qualité de la présentation et du niveau d'implication des étudiants.*

- **Fonctionnalité et efficacité** du système mis en place.
- **Sécurité du chiffrement** et du transfert.
- **Automatisation complète et fiabilité du processus.**
- **Qualité** du code et de la **documentation.**
- **Originalité et pertinence** des solutions proposées.

## MODALITÉS DU PROJET

- Durée de rendu: **12/11/25**
- Ressources :
  - Ressources documentaires numérique en ligne
  - Cours sur la Plateforme 360Learning
  - Documentation officielle MariaDB et OpenSSL
  - Références Bash
  - Bibliothèque ENI
- Objectifs et livrables :
  - Code : Le code ou script permettant d'automatiser la sauvegarde, le chiffrement, le transfert et la vérification d'intégrité.
  - Documentation : Un fichier décrivant les choix techniques, les étapes d'installation et d'utilisation du système.
  - Présentation : Un rapport expliquant les démarches, les technologies choisies, et comment le système répond aux besoins exprimés.

## Table des matières

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>1. Introduction .....</b>                                     | <b>09</b> |
| 1.1. Contexte du projet .....                                    | 09        |
| 1.2. Objectifs généraux et rappel des attendus .....             | 10        |
| 1.3. Présentation rapide de la solution proposée .....           | 11        |
| <b>2. Organisation du projet .....</b>                           | <b>12</b> |
| 2.1 Méthodologie de travail (sprints) .....                      | 12        |
| 2.2 Outils de gestion de projet utilisés (Jira) .....            | 13        |
| 2.3 Planning prévisionnel des tâches (Gantt) .....               | 14        |
| <b>3. Conception du système .....</b>                            | <b>15</b> |
| 3.1. Architecture générale du système .....                      | 15        |
| 3.2. Diagramme de fonctionnement global .....                    | 17        |
| 3.3. Description des modules .....                               | 17        |
| Exportation de la base de données .....                          | 17        |
| Chiffrement du dump .....                                        | 17        |
| Transfert sécurisé vers le serveur distant .....                 | 17        |
| Vérification d'intégrité .....                                   | 17        |
| Restauration de la base à partir de la sauvegarde chiffrée ..... | 18        |
| Automatisation .....                                             | 18        |
| 3.4. Choix technologiques et justification .....                 | 18        |
| <b>4. Mise en oeuvre technique .....</b>                         | <b>21</b> |
| 4.1. Environnement de test .....                                 | 21        |
| 4.2. Structure des scripts / fichiers du projet .....            | 23        |
| <b>Conclusion.....</b>                                           | <b>24</b> |
| <b>Ressources documentaires et références utilisés.....</b>      | <b>25</b> |
| <b>Annexes.....</b>                                              | <b>26</b> |

## **Projet Cybersecurite-AP1 — Mise en place d'un système de sauvegarde automatisée et sécurisée d'une base de données**



Projet Cybersecurite-AP1 — SN2 2025-2026  
Atelier Cybersécurité

# Table des Annexes

Le dossier de projet suivant contient plusieurs annexes regroupant plusieurs images, illustrations et schémas.

Structure du projet ..... 27

Schéma global du fonctionnement du script ..... 28

Liste des principales commandes du CLI ..... 29

## Avant propos

L'ensemble des scripts, fichiers et éléments nécessaires à la reproduction de ce projet est disponible sur GitHub :

<https://github.com/kaelianbaudelet/projet-cybersecurite-AP1>.

Ce projet a été conçu dans un objectif **pédagogique et technique**, visant à illustrer la mise en place d'un **système de sauvegarde automatisé et sécurisé**.



## 1. Introduction

### 1.1. Contexte du projet

Dans le cadre de l'atelier de cybersécurité, ce projet **consistera à concevoir et mettre en place un système de sauvegarde automatisée et sécurisée d'une base de données MariaDB**. L'objectif sera d'assurer la **confidentialité, l'intégrité et la disponibilité** des données, trois principes fondamentaux de la sécurité informatique.

Le projet permettra d'aborder de manière pratique plusieurs notions essentielles de la cybersécurité :

- **Chiffrement des données** pour garantir la protection des informations sensibles lors du transfert ou du stockage.
- **Transfert sécurisé** des fichiers de sauvegarde vers un serveur distant afin de limiter les risques d'accès non autorisé.
- **Vérification de l'intégrité** des sauvegardes pour s'assurer qu'aucune corruption ou modification n'a eu lieu.
- **Automatisation des processus** afin de réduire les erreurs humaines et garantir la disponibilité des données.

Le système développé **devra fonctionner de manière autonome et reproductible**, offrant une approche concrète de la sécurisation des informations et de l'administration sécurisée des infrastructures, tout en intégrant des bonnes pratiques de scripting et de DevOps adaptées à la cybersécurité.

## 1.2. Objectifs généraux et rappel des attendus

Les objectifs pédagogiques et techniques de ce projet sont centrés sur la mise en place d'un **système de sauvegarde automatisée et sécurisée d'une base de données MariaDB**, tout en respectant les bonnes pratiques de cybersécurité et d'automatisation.

### Objectifs pédagogiques :

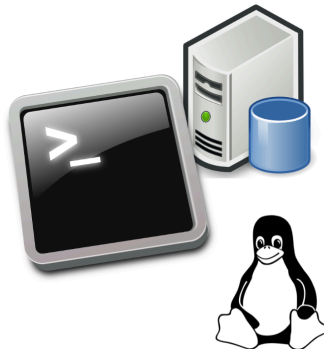
- Comprendre les principes de **confidentialité, intégrité et disponibilité** des données dans un contexte réel.
- Apprendre à concevoir un **processus de sauvegarde automatisé**, incluant l'exportation, le chiffrement, le transfert sécurisé et la vérification d'intégrité.
- Expérimenter l'utilisation de protocoles sécurisés pour la transmission de fichiers sensibles.
- Documenter de manière claire et structurée l'ensemble des étapes de conception, de test et de validation du système.
- Travailler en **méthode SCRUM**.

### Objectifs techniques :

- Développer des scripts permettant d'**exporter automatiquement la base MariaDB** et de générer des dumps fiables.
- Implémenter un **chiffrement robuste** pour sécuriser les fichiers de sauvegarde.
- Mettre en place un **transfert sécurisé vers un serveur distant** et automatiser ce processus.
- Créer un mécanisme de **vérification d'intégrité** des sauvegardes pour s'assurer qu'aucune altération ne survient.
- Développer une procédure de **restauration automatisée** à partir des fichiers chiffrés.
- Garantir que le système fonctionne de manière autonome et répétable, avec un minimum d'intervention humaine.

L'objectif final est d'obtenir un **système fiable, sécurisé et automatisé**, capable de protéger les données d'une base MariaDB tout en facilitant leur restauration en cas de besoin. Ce projet permettra de mettre en pratique des compétences concrètes en cybersécurité, en scripting et en automatisation de processus critiques.

### 1.3. Présentation rapide de la solution proposée



Pour répondre aux objectifs de ce projet, la solution choisie consiste à développer une **interface en ligne de commande (CLI) entièrement automatisée**, utilisant le langage **Bash**, reconnu pour sa puissance dans le scripting système et l'automatisation de tâches sur les environnements Linux.

La CLI sera le **point central de l'architecture**, permettant de contrôler l'ensemble du processus de sauvegarde et de restauration. Elle offrira une interface simple mais complète, où l'utilisateur pourra :

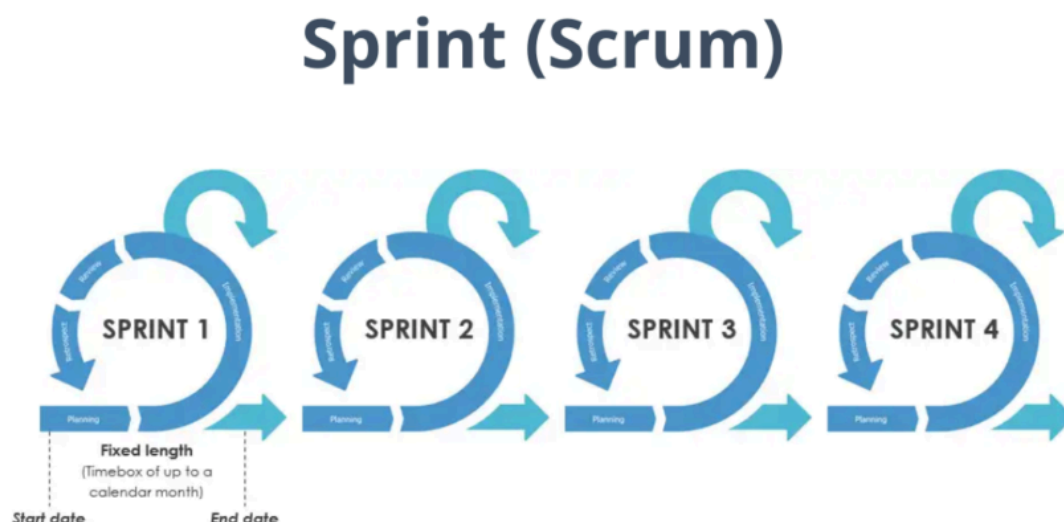
- **Définir les paramètres de la base de données** (nom de la base, utilisateur, mot de passe, hôte) et du serveur distant.
- **Lancer la création d'un dump automatisé** de la base MariaDB, garantissant la cohérence et la complétude des données exportées.
- **Chiffrer les fichiers de sauvegarde** afin de protéger les informations sensibles lors du stockage ou du transfert.
- **Transférer les fichiers vers un serveur distant** via un protocole sécurisé, en assurant que le processus est entièrement automatisé et répétable.
- **Vérifier l'intégrité des sauvegardes** en comparant les fichiers locaux et distants pour détecter toute corruption ou modification.
- **Restaurer la base de données** à partir du fichier chiffré stocké sur le serveur distant, avec un minimum d'intervention manuelle.

L'architecture choisie repose sur une **approche modulaire** : chaque fonction de la CLI correspond à une étape distincte du processus (export, chiffrement, transfert, intégrité, restauration), ce qui permet de maintenir facilement le code et d'ajouter des fonctionnalités futures si nécessaire. Cette modularité assure également la **réutilisabilité** et la **robustesse** du système.

En utilisant Bash, la solution tire parti de la **flexibilité du scripting**, de la compatibilité avec la majorité des systèmes Linux, et de la possibilité d'intégrer des commandes système et des utilitaires existants (comme mysqldump, scp, openssl) pour automatiser l'ensemble du workflow. Cette approche combine ainsi **simplicité, efficacité et sécurité**, tout en offrant un outil pratique et autonome pour la gestion des sauvegardes de bases de données MariaDB.

## 2. Organisation du projet

### 2.1 Méthodologie de travail (sprints)



Pour structurer le développement du système de sauvegarde automatisée et sécurisée, le projet a été organisé selon une méthodologie basée sur des sprints. Cette approche permet de découper le projet en périodes de travail courtes et ciblées, durant lesquelles des objectifs précis doivent être atteints. Chaque sprint correspond à un ensemble de tâches clairement définies, allant de l'étude initiale des besoins jusqu'à la mise en place des fonctionnalités et à la validation finale du système.

L'utilisation de sprints a été particulièrement adaptée à ce projet car elle facilite la gestion progressive et contrôlée des différentes étapes : l'étude et l'analyse du cahier des charges, le choix des outils et technologies, le développement des modules du CLI, les tests et la documentation. Cette méthode permet également d'identifier rapidement les éventuels blocages ou difficultés techniques, et d'ajuster le planning de manière réactive.

En adoptant une démarche itérative, chaque module du CLI a été développé, testé et validé avant de passer à l'étape suivante. Cette organisation garantit non seulement un développement plus sûr et plus efficace, mais aussi une meilleure traçabilité des décisions prises tout au long du projet. Elle reflète également les pratiques professionnelles actuelles en matière de gestion de projets logiciels, notamment dans le domaine de la cybersécurité où la rigueur et le suivi méthodique sont essentiels.

2.2 Outils de gestion de projet utilisés (Jira)



Pour organiser le projet et suivre l'avancement des tâches, l'outil Jira a été utilisé. Chaque tâche identifiée dans le planning prévisionnel a été créée dans Jira, avec ses informations détaillées : description, responsable, date de début et date de fin.

Par exemple :

- Les étapes initiales comme l'analyse du cahier des charges (27/10) et l'étude des outils et choix technologiques (28-30/10) ont été planifiées et suivies dans Jira pour s'assurer qu'elles soient terminées avant de passer à la conception.
- La conception et le développement des modules du CLI (03-09/11) ont été découpés en tickets correspondant à chaque fonctionnalité : export de la base, chiffrement, transfert sécurisé, vérification d'intégrité, automatisation et restauration. Jira a permis de suivre la progression de chaque module et de vérifier que le développement respectait le calendrier prévu.
- Les tâches liées aux tests, validation et documentation (03-10/11) ont également été suivies dans Jira, garantissant que chaque étape de vérification et de rédaction du rapport était réalisée dans les temps.

Ainsi, Jira a servi de tableau de bord centralisé, permettant de visualiser rapidement l'état d'avancement du projet, de contrôler le respect des délais et de coordonner toutes les étapes planifiées dans le Gantt prévisionnel.

| Tickets                                                                                    | Personne assignée | Rapporteur       | Priorité | État       | Résolution | Création             | Date d'échéance |
|--------------------------------------------------------------------------------------------|-------------------|------------------|----------|------------|------------|----------------------|-----------------|
| <input checked="" type="checkbox"/> CYB-30 Préparation de la présentation et de la démo... | Kaelian BAUDELET  | Kaelian BAUDELET | High     | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:36 | 10 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-29 Rédaction du rapport de projet                  | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:36 | 10 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-28 Validation du système                           | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:36 | 09 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-27 Tests                                           | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:36 | 10 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-26 Élaboration du plan de tests                    | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:36 | 08 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-25 Mise en place d'un système de restauration d... | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:36 | 09 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-24 Mise en place d'une logique d'automatisation    | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:36 | 09 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-23 Mise en place de la vérification d'intégrité    | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:35 | 09 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-22 Mise en place du transfert sécurisé             | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:35 | Aucun           |
| <input checked="" type="checkbox"/> CYB-21 Mise en place du chiffrement du dump            | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:35 | 06 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-20 Mise en place de l'export base de données       | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:35 | 05 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-19 Structure du projet CLI                         | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:35 | 04 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-18 Architecture du système                         | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:35 | 03 nov. 2025    |
| <input checked="" type="checkbox"/> CYB-17 Mise en place du dépôt Git                      | Kaelian BAUDELET  | Kaelian BAUDELET | Highest  | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:34 | 27 oct. 2025    |
| <input checked="" type="checkbox"/> CYB-16 Planification du projet                         | Kaelian BAUDELET  | Kaelian BAUDELET | Highest  | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:34 | 31 oct. 2025    |
| <input checked="" type="checkbox"/> CYB-15 Spécifications techniques                       | Kaelian BAUDELET  | Kaelian BAUDELET | Medium   | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:34 | 31 oct. 2025    |
| <input checked="" type="checkbox"/> CYB-14 Étude des outils et choix de la techno          | Kaelian BAUDELET  | Kaelian BAUDELET | Highest  | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:34 | 30 oct. 2025    |
| <input checked="" type="checkbox"/> CYB-13 Analyse du cahier des charges                   | Kaelian BAUDELET  | Kaelian BAUDELET | Highest  | TERMINÉ(E) | Terminé    | 31 oct. 2025 à 19:34 | 27 oct. 2025    |



### 3. Conception du système

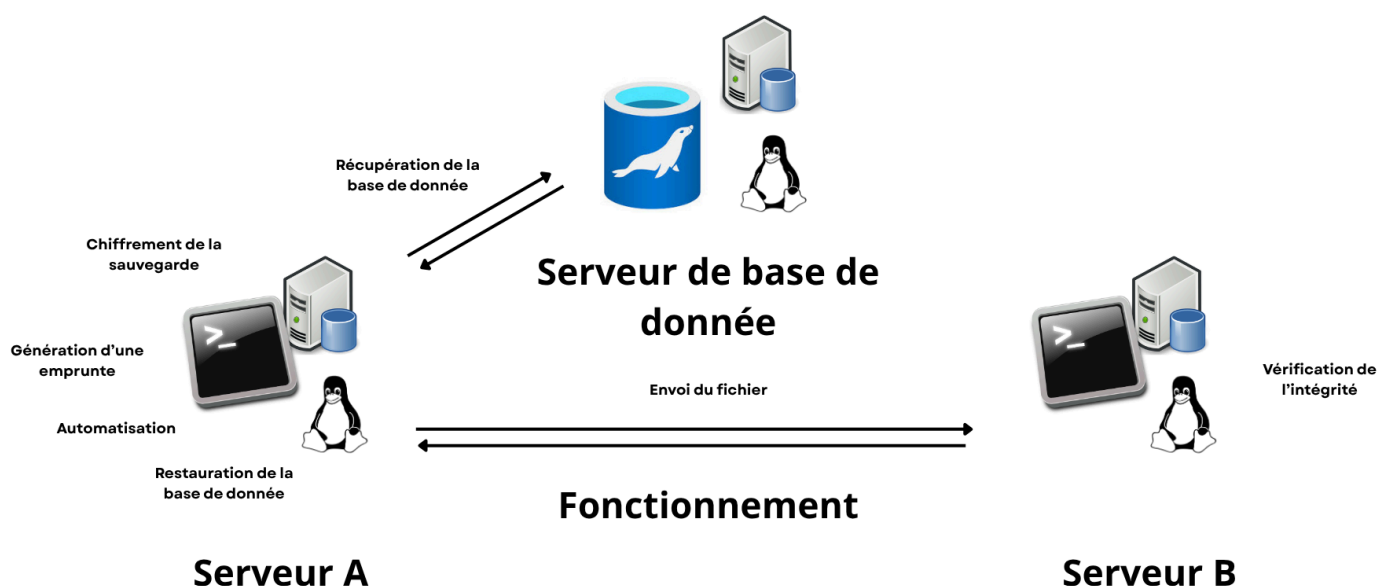
#### 3.1. Architecture générale du système

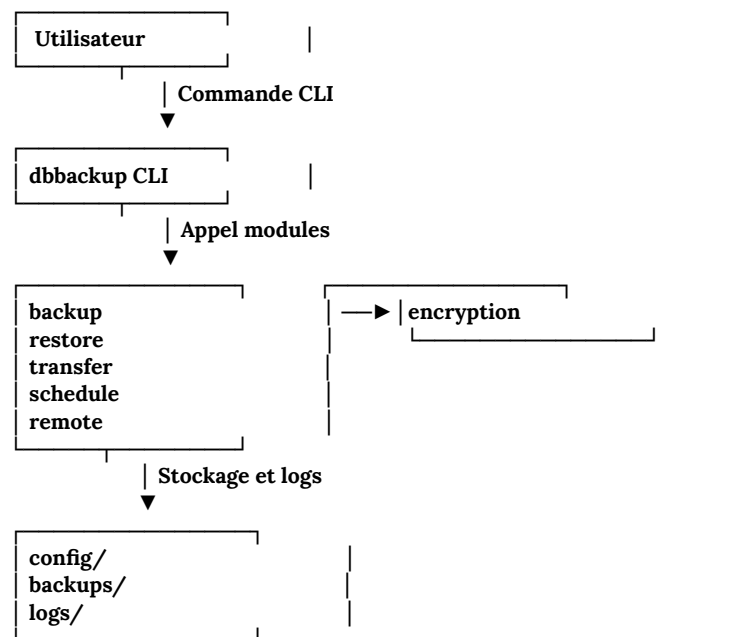
Le système DBBackup CLI est conçu comme une suite d'outils modulaires en Bash permettant la gestion complète des sauvegardes de bases de données MariaDB/MySQL. Il repose sur une architecture en couches :

- **Interface CLI** : point d'entrée dbbackup pour exécuter les commandes principales (backup, restore, schedule, remote, config).
- **Modules fonctionnels** : scripts Bash séparés (backup.sh, restore.sh, encryption.sh, transfer.sh, schedule.sh, remote.sh) qui réalisent chacune des opérations spécifiques.
- **Configuration et stockage** : dossier config/ pour les paramètres, clés de chiffrement et serveurs distants, et dossier backups/ pour les sauvegardes locales.
- **Transfert et sécurité** : intégration de protocoles sécurisés (SFTP/SSH) et chiffrement AES-256 pour assurer confidentialité et intégrité des données.

Cette architecture modulaire facilite la maintenance, la réutilisation des composants et l'extension vers de nouveaux services (par exemple d'autres types de bases de données ou protocoles de transfert).

#### 3.2. Diagramme de fonctionnement global





Le schéma ci-dessus illustre le fonctionnement global de DBBackup CLI et la manière dont ses différents modules interagissent pour garantir la sauvegarde, la protection et la restauration des bases de données. L'utilisateur lance une commande via l'interface CLI, qui sert de point d'accès central pour toutes les actions. Selon l'opération choisie, le module correspondant prend le relais et exécute sa tâche spécifique.

Le module de sauvegarde crée une copie complète de la base de données, qui est ensuite sécurisée par le module de chiffrement afin de préserver sa confidentialité. Si nécessaire, le module de transfert se charge de déplacer la sauvegarde vers un emplacement distant, de façon autonome et transparente pour l'utilisateur. La fiabilité des fichiers est assurée par le module de vérification d'intégrité, tandis que le module de planification permet de gérer automatiquement l'exécution des sauvegardes selon des horaires prédéfinis. Enfin, le module de restauration récupère les fichiers existants et remet la base dans son état initial, en s'appuyant sur le travail effectué par les autres composants.

Dans l'ensemble, le diagramme reflète une organisation claire et modulaire : chaque module a sa responsabilité propre, fonctionne de manière autonome, tout en s'intégrant dans un flux cohérent qui va de l'initiation de la commande par l'utilisateur jusqu'à la restauration complète de la base. Cette structure rend le système à la fois robuste, fiable et simple à maintenir, en garantissant que chaque étape du processus, sauvegarde, sécurisation, transfert, vérification et restauration est exécutée correctement.



### 3.3 Description des modules

Le système DBBackup CLI est conçu de manière modulaire, chaque composant ayant un rôle précis pour assurer la sauvegarde, la sécurisation et la restauration des bases de données.

#### 3.3.1. Exportation de la base de données

Le module d'exportation est responsable de la création de la copie complète de la base de données. Il utilise l'outil natif mysqldump pour générer un dump SQL précis et complet. Cette approche permet d'extraire toutes les tables et données de manière fiable, en respectant la structure exacte de la base. Le choix de mysqldump est justifié par sa compatibilité native avec MariaDB/MySQL pour les sauvegardes complètes. Cette étape constitue le point de départ du processus de sauvegarde, car la qualité du dump détermine la fiabilité de toutes les étapes suivantes.

#### 3.3.2. Chiffrement du dump

Une fois le dump créé, le module de chiffrement prend le relais. Il utilise AES-256-CBC, un algorithme reconnu pour sa robustesse et sa compatibilité avec OpenSSL, afin de sécuriser le fichier. La clé de chiffrement est stockée dans config/encryption.key, garantissant que le contenu reste confidentiel que ce soit pour un stockage local ou pour un transfert distant. Le chiffrement protège les données sensibles contre tout accès non autorisé, ce qui est crucial lorsque les sauvegardes quittent le serveur principal.

#### 3.3.3. Transfert sécurisé vers le serveur distant

Ensuite, le module de transfert est appelé pour déplacer les fichiers vers un serveur distant. Il utilise des protocoles sécurisés SSH/SFTP avec SCP pour garantir la confidentialité et l'intégrité des fichiers lors du transit. Cette étape est automatisée pour fonctionner de manière transparente pour l'utilisateur et permet de disposer de sauvegardes redondantes sur plusieurs emplacements, renforçant ainsi la résilience face aux pertes locales ou incidents matériels. Le choix de SSH/SCP repose sur leur sécurité éprouvée et leur compatibilité avec la majorité des serveurs Linux, tout en étant simple à intégrer dans un script Bash.

#### 3.3.4. Vérification d'intégrité

Le module de vérification d'intégrité contrôle que les fichiers restent fiables et intacts à chaque étape du processus. Il génère une somme de contrôle SHA-256 via la commande shasum -a 256 pour chaque sauvegarde, puis la compare lors de la restauration ou du transfert. Cette vérification assure que les données n'ont pas été corrompues ou altérées, offrant une garantie supplémentaire sur la fiabilité des sauvegardes.

### 3.3.5. Restauration de la base à partir de la sauvegarde chiffrée

Lorsque l'utilisateur souhaite restaurer une base, le module de restauration récupère le fichier existant, le déchiffre via OpenSSL avec la clef de chiffrement dans `encryption.key`, et le réinjecte dans la base cible à l'aide de `mysql`. Il peut vérifier l'intégrité du fichier avant de restaurer, afin de garantir que la base restaurée correspond exactement à l'original. Cette approche technique assure une restauration fiable et sécurisée, tout en exploitant les mêmes standards de compatibilité que ceux utilisés pour la sauvegarde.

### 3.3.6. Automatisation

Enfin, le module de planification permet d'automatiser l'ensemble du processus grâce à `cron`, qui déclenche les sauvegardes, transferts et vérifications selon des horaires définis dans `config/schedules.conf`. Cette automatisation garantit la régularité et la continuité des sauvegardes sans intervention manuelle, tout en permettant de définir différentes fréquences selon l'importance des bases. L'utilisation de `cron` est justifiée par sa fiabilité et son intégration native dans les systèmes Unix/Linux, ce qui en fait une solution simple et efficace pour gérer les tâches planifiées.

Dans l'ensemble, les modules interviennent de manière séquentielle et coordonnée : la création du dump avec `mysqldump` lance le processus, le chiffrement AES-256-CBC et le transfert via SSH/SCP assurent sécurité et disponibilité, la vérification SHA-256 garantit la fiabilité, et la restauration clôt le cycle. Cette organisation modulaire et fluide permet d'obtenir un système robuste, clair, sécurisé et facile à maintenir.

### 3.4. Choix technologiques et justification

Le projet **DBBackup CLI** repose sur des choix techniques motivés par la simplicité, la compatibilité et l'efficacité dans la gestion des sauvegardes.



Le langage principal utilisé est **Bash**, car il est directement disponible sur la majorité des systèmes Unix/Linux et permet d'exécuter facilement des commandes système, de gérer les fichiers et de coordonner des opérations successives comme la création de dumps, le chiffrement, le transfert et la vérification d'intégrité. Cette simplicité évite d'ajouter des dépendances complexes et facilite l'automatisation avec des outils natifs comme **cron**.



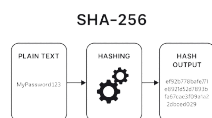
Pour l'exportation des bases, le projet utilise **mysqldump**, l'outil natif de MariaDB/MySQL. Il permet de générer des dumps complets et fiables, avec toutes les tables et données nécessaires, et est compatible avec tous les environnements MySQL/MariaDB. Les fichiers SQL générés sont ensuite compressés avec **gzip**, afin de réduire leur taille et faciliter leur stockage ou transfert.



Le chiffrement est réalisé via **OpenSSL** avec **AES-256-CBC**, ce qui assure la confidentialité des données sensibles. La clé est stockée dans un fichier séparé (config/encryption.key), ce qui permet de gérer la sécurité indépendamment du script principal. Le format final des fichiers est .sql.gz.enc, combinant SQL, compression et chiffrement pour obtenir des sauvegardes compactes et sécurisées.



Pour le transfert vers un serveur distant, le projet utilise **SCP** via **SSH**. Ce protocole est simple à intégrer dans un script Bash et garantit que les fichiers sont transférés de manière sécurisée. Le choix de SCP repose sur sa compatibilité universelle et sa simplicité pour envoyer des fichiers uniques ou des archives chiffrées, sans nécessiter de configuration complexe.



Enfin, le module de vérification d'intégrité utilise des **hashs SHA-256**, calculés avec la commande `shasum -a 256`. Cette vérification permet de détecter toute altération ou corruption des fichiers à chaque étape, que ce soit après la création du dump, le chiffrement ou le transfert, avant toute restauration. L'utilisation de SHA-256 est simple à intégrer et suffisamment robuste pour s'assurer de l'intégrité des sauvegardes.

## 4. Mise en oeuvre technique

### 4.1 Environnement de test

The screenshot shows a code editor with the following content:

```

dbbackup -- Projet AP1
$ encryption.sh $ remote.sh $ restore.sh $ schedule.sh $ transfer.sh ... $ dbbackup x

PROJECT AP1
  dbbackup-cli
  backups
  config
  remotes
  dbbackup.conf
  encryption.key
  schedules.conf
  transfer.conf
  lib
  backup.sh
  config.sh
  encryption.sh
  remote.sh
  restore.sh
  schedule.sh
  transfer.sh
  logs
  dbbackup
  README.md
  dbbackup-cli.zip
  exemples_de_cmd
  Sujet-Cybersecu-AP1.pdf

dbbackup-cli > lib > $ transfer.sh
237 test_remote_connection() {
238     if [[ "${REMOTE_AUTH_METHOD}" == "key" ]]; then
239         if [[ ! -f "${REMOTE_SSH_KEY}" ]]; then
240             return 1
241         fi
242         ssh_cmd="ssh -p ${REMOTE_PORT} -i ${REMOTE_SSH_KEY} \
243             -o StrictHostKeyChecking=no \
244             -o UserKnownHostsFile=/dev/null \
245             -o ConnectTimeout=10 \
246             -o LogLevel=ERROR"
247     else
248         if ! command -v sshpass > /dev/null; then
249             log_error "sshpass introuvable"
250             return 1
251         fi
252         ssh_cmd="sshpass -p ${REMOTE_PASSWORD} ssh -p ${REMOTE_PORT} \
253             -o StrictHostKeyChecking=no \
254             -o UserKnownHostsFile=/dev/null \
255             -o ConnectTimeout=10 \
256             -o LogLevel=ERROR"
257     fi
258     if $(ssh_cmd) "${REMOTE_USER}@${REMOTE_HOST}" "echo 'Connexion réussie'"
259     log_success "Test de connexion distante réussi"
260     return 0
261 else
262     log_error "Échec du test de connexion distante"
263 fi
264
265 kaelian@MacBook-Pro-de-Kaelian dbbackup-cli % ./dbbackup backup list
Sauvegardes locales dans /Users/kaelian/Desktop/Projet AP1/dbbackup-cli/backups :
NOM                                DATE DE CRÉATION                                FICHIER
-----
ma_base_20251112_114020             2025-11-12 11:44:20                             ma_base_20251112_114020.sql.gz.enc
Kaelian@MacBook-Pro-de-Kaelian dbbackup-cli %
  
```

Pour le développement et les tests du système de sauvegarde automatisée, l'environnement principal choisi est mon **MacBook personnel**. Ce choix s'explique par la compatibilité native avec le langage **Bash**. Il a fallu ajouter simplement les utilitaires système nécessaires comme `mysqldump` pour l'export des bases de données, `openssl` pour le chiffrement et `scp` pour le transfert sécurisé des fichiers. L'environnement local permet ainsi de créer et de tester rapidement des scripts tout en offrant une grande flexibilité pour ajuster les paramètres et dépanner en temps réel.

The screenshot shows a virtual machine environment with the following content:

```

MACHINES VIRTUELLES
  ARM Ubuntu 64 bits S...
  ARM Windows 11 64 bits
  Projet site web

ARM Ubuntu 64 bits Server 24.04.3 (Cours BRKI)

Ubuntu 24.04.3 LTS ubuntu tty1
ubuntu login: af1
Password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-87-generic aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

af1@ubuntu:~$ cd /backup
af1@ubuntu:/backup$ ls -la
total 12K
-rw-r--r-- 1 af1 af1 4K Nov 12 11:40 ma_base_20251112_112722.sql.gz.enc
-rw-r--r-- 1 af1 af1 4K Nov 12 11:40 ma_base_20251112_114031.sql.gz.enc
af1@ubuntu:/backup$

- zsh
kaelian@MacBook-Pro-de-Kaelian dbbackup-cli % ./dbbackup backup list --remote prodserverclear
[2025-11-13 12:59:03] [ERREUR] Serveur distant 'prodserverclear' introuvable. Utilisez 'dbbackup remote list' pour vérifier.
kaelian@MacBook-Pro-de-Kaelian dbbackup-cli % ./dbbackup backup list --remote prodserver

Sauvegardes distantes pour 'prodserver' (af1@172.16.252.142:/backup) :
NOM                                DATE DE CRÉATION                                FICHIER
-----
ma_base_20251112_114031             2025-11-12 10:40:32                             ma_base_20251112_114031.sql.gz.enc
ma_base_20251112_112722             2025-11-12 10:27:22                             ma_base_20251112_112722.sql.gz.enc
kaelian@MacBook-Pro-de-Kaelian dbbackup-cli %
  
```

Afin de simuler un serveur distant et de valider le processus complet de transfert sécurisé et de restauration, une **machine virtuelle locale** a été configurée. Cette VM joue le rôle du serveur de sauvegarde distant, recevant les fichiers exportés et chiffrés. Cette approche permet de tester intégralement le fonctionnement du système sans avoir besoin d'un serveur physique externe, tout en reproduisant les contraintes liées à un transfert réseau réel. La VM dispose de MariaDB et d'un espace dédié pour stocker les sauvegardes, ainsi que des droits configurés spécifiquement pour l'utilisateur recevant les fichiers, garantissant ainsi la sécurité et l'isolation des tests.

L'ensemble des tests a été conçu pour valider le processus de bout en bout : de l'exportation de la base sur le MacBook, au chiffrement, à l'envoi sur la VM distante et à la restauration sur cette même VM. Cette configuration permet également d'évaluer le système face aux erreurs potentielles, comme des interruptions de transfert ou des fichiers corrompus, et de s'assurer que les mécanismes de vérification d'intégrité fonctionnent correctement.

## 4.2 Structure des scripts / fichiers du projet

Le projet a été conçu selon une **architecture modulaire et hiérarchisée**, facilitant la maintenance et l'évolution future du système. Le script principal `cli_backup.sh` sert d'interface CLI, permettant à l'utilisateur de lancer toutes les fonctionnalités du système depuis une seule commande. Ce script coordonne les différents modules, en appelant chaque fonctionnalité dans l'ordre nécessaire pour garantir l'automatisation complète.

|                        |                                                                       |
|------------------------|-----------------------------------------------------------------------|
| dbbackup-cli           |                                                                       |
| ├── backups            | Dossier où sont enregistré les sauvegardes locales                    |
| ├── config             | Dossier des fichiers de configuration                                 |
| │   ├── dbbackup.conf  | Fichier de configuration global du CLI                                |
| │   ├── encryption.key | Fichier contenant la clef d'encryption                                |
| │   ├── remotes        | Dossier contenant les configuration des serveurs distants             |
| │   ├── schedules.conf | Fichier de configuration des sauvegardes planifiées                   |
| │   └── transfer.conf  | Fichier de configuration des transfert à distance de sauvegardes      |
| ├── dbbackup           | Script Principal / Point d'entrée du CLI                              |
| ├── lib                | Modules du projet                                                     |
| │   ├── backup.sh      | Module Bash chargé des sauvegardes                                    |
| │   ├── config.sh      | Module Bash chargé de la configuration                                |
| │   ├── encryption.sh  | Module Bash chargé de l'encryption des sauvegardes                    |
| │   ├── remote.sh      | Module Bash chargé de la gestion des serveurs distants de sauvegardes |
| │   ├── restore.sh     | Module Bash chargé de la restauration des sauvegardes                 |
| │   ├── schedule.sh    | Module Bash chargé de la planification des sauvegardes                |
| │   └── transfer.sh    | Module Bash chargé de l'envoi à distance des sauvegardes              |
| └── logs               | Dossier des fichiers journaux                                         |

Le répertoire **modules/** contient les scripts spécialisés pour chaque étape du processus :

- `export_db.sh` gère l'export de la base MariaDB vers un fichier SQL.
- `encrypt_dump.sh` effectue le chiffrement du fichier de sauvegarde avec OpenSSL, garantissant la confidentialité des données.
- `transfer.sh` assure le transfert sécurisé du fichier vers le serveur distant via SCP ou SFTP.
- `verify_integrity.sh` compare le fichier local et le fichier distant pour s'assurer que la sauvegarde n'a pas été corrompue pendant le transfert.
- `restore_db.sh` permet la restauration de la base à partir du dump chiffré sur le serveur distant.

Le répertoire **config/** contient les fichiers de configuration définissant les paramètres globaux, tandis que le répertoire **logs/** centralise tous les journaux d'exécution, permettant de suivre le déroulement du processus et de détecter rapidement toute anomalie. Enfin, le fichier **README.md** documente l'ensemble du projet, avec les instructions d'installation, de configuration et d'utilisation de la CLI.

Cette organisation modulaire permet d'assurer une **clarté maximale du code**, de tester facilement chaque module individuellement et de garantir que le système reste **autonome, sécurisé et facilement extensible** pour de futures améliorations ou adaptations à d'autres environnements.

## Conclusion

Ce projet avait pour objectif de concevoir un script Bash capable d'automatiser la sauvegarde d'une base de données MariaDB tout en garantissant sa sécurité et sa fiabilité. L'ensemble des étapes clés a été respecté : l'exportation de la base de données, le chiffrement du fichier de sauvegarde, le transfert sécurisé vers un serveur distant, la vérification de l'intégrité du fichier reçu, ainsi que la mise en place d'un mécanisme de restauration.

Le système a été entièrement automatisé afin de fonctionner de manière autonome à intervalles réguliers, sans nécessiter d'intervention humaine.

Ce projet m'a permis de consolider mes compétences en scripting Bash, en administration de bases de données et en sécurité des systèmes. Il m'a également apporté une meilleure compréhension des principes d'automatisation et de fiabilité applicables aux environnements de production.

En conclusion, la solution développée répond pleinement aux objectifs fixés : elle permet d'assurer une **sauvegarde automatique, chiffrée et sécurisée** des données.



## Ressources documentaires et références utilisés

### Documentation utilisées

- Documentation de MariaDB – <https://mariadb.com/kb/en/documentation/>
- Documentation de OpenSSL – <https://www.openssl.org/docs/>
- Documentation de Bash GNU – <https://www.gnu.org/software/bash/manual/>
- Documentation MAN de CRON (Planificateur de tâches Linux) – <https://man7.org/linux/man-pages/man8/cron.8.html>
- Documentation MAN de SCP et SFTP – <https://man7.org/linux/man-pages/man1/scp.1.html>
- Documentation MAN de SHA256SUM / intégrité – <https://man7.org/linux/man-pages/man1/sha256sum.1.html>

### Articles techniques et tutoriels consultés

- IT Connect - Sauvegarder et restaurer une base de données MySQL – <https://www.it-connect.fr/comment-sauvegarder-et-restaurer-une-base-de-donnees-avec-mysqldump/>
- StackOverflow - File Encryption/Decryption with AES using Linux – <https://stackoverflow.com/questions/41141794/file-encryption-decryption-with-aes-using-linux>
- IT Connect - Crontab et Cron : guide d'utilisation sous Linux – <https://www.it-connect.fr/automatiser-des-taches-avec-crontab/>
- Linux : vérifier l'intégrité d'un fichier via son empreinte (SHA256, SHA1, MD5...) – <https://lecrabeinfo.net/tutoriels/verifier-integrite-calculer-empreinte-checksum-sha256-sha1-md5-fichier-linux/>

### Ressources pédagogiques et cours utilisés

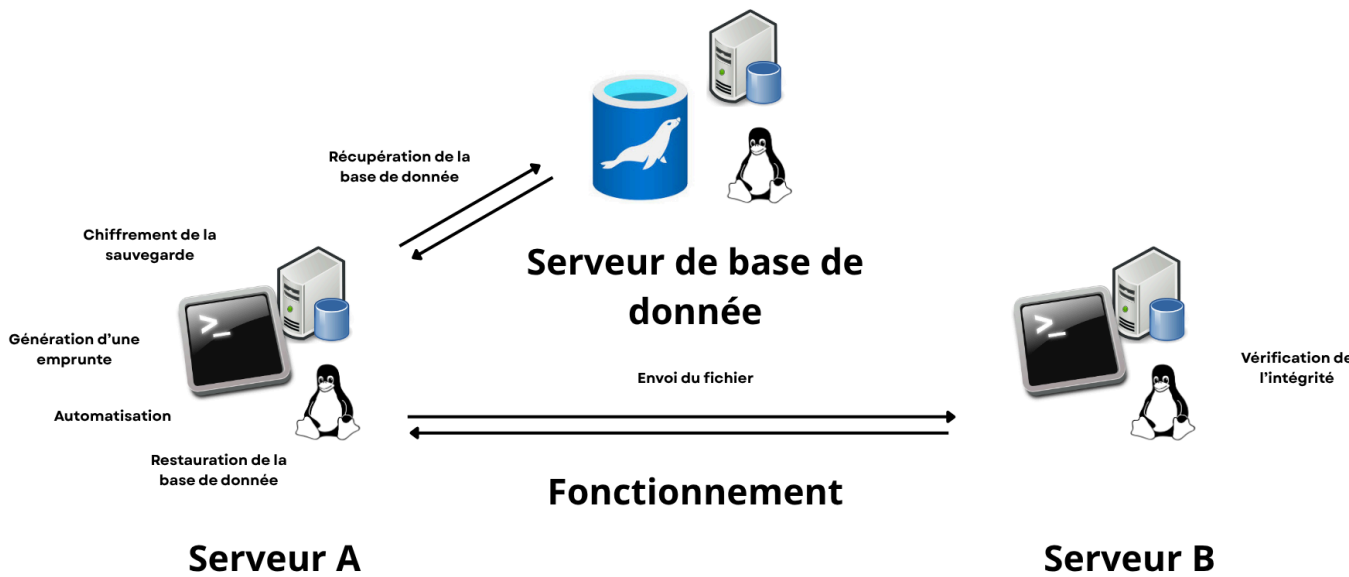
- Cours EPSI / Module **Atelier Environnement Linux ADMIN POSTE TRAVAIL INI2 (LINE 312)**
- Cours EPSI / Module **RESE 306 : Admin poste Client Gestion des accès (202526)**
- LinkedInLearning / Découvrir l'écriture de scripts en Bash – <https://www.linkedin.com/learning/decouvrir-l-ecriture-de-scripts-en-bash/bienvenue-dans-decouvrir-l-ecriture-de-scripts-en-bash?u=43271628>

## **Annexes**

## Structure du projet

|                        |                                                                       |
|------------------------|-----------------------------------------------------------------------|
| dbbackup-cli           |                                                                       |
| ├── backups            | Dossier où sont enregistré les sauvegardes locales                    |
| ├── config             | Dossier des fichiers de configuration                                 |
| │   ├── dbbackup.conf  | Fichier de configuration global du CLI                                |
| │   ├── encryption.key | Fichier contenant la clef d'encryption                                |
| │   ├── remotes        | Dossier contenant les configuration des serveurs distants             |
| │   ├── schedules.conf | Fichier de configuration des sauvegardes planifiées                   |
| │   └── transfer.conf  | Fichier de configuration des transfert à distance de sauvegardes      |
| ├── dbbackup           | Script Principal / Point d'entrée du CLI                              |
| ├── lib                | Modules du projet                                                     |
| │   ├── backup.sh      | Module Bash chargé des sauvegardes                                    |
| │   ├── config.sh      | Module Bash chargé de la configuration                                |
| │   ├── encryption.sh  | Module Bash chargé de l'encryption des sauvegardes                    |
| │   ├── remote.sh      | Module Bash chargé de la gestion des serveurs distants de sauvegardes |
| │   ├── restore.sh     | Module Bash chargé de la restauration des sauvegardes                 |
| │   ├── schedule.sh    | Module Bash chargé de la planification des sauvegardes                |
| │   └── transfer.sh    | Module Bash chargé de l'envoi à distance des sauvegardes              |
| └── logs               | Dossier des fichiers journaux                                         |

Schéma global du fonctionnement du script



## Liste des principales commandes du CLI

### dbbackup backup

Créer une sauvegarde de base de données, compressée, chiffrée et optionnellement transférée vers un serveur distant.

Options utiles :

- `-h|--host hôte` MariaDB (defaut localhost).
- `-P|--port port` MariaDB (defaut 3306).
- `-u|--user utilisateur` MariaDB (obligatoire).
- `-p|--password mot de passe` (obligatoire).
- `-d|--database base` à sauvegarder (obligatoire).
- `-o|--output dossier local` (defaut backups/).
- `--encrypt / --no-encrypt` active ou désactive le chiffrement (defaut actif).
- `--transfer / --no-transfer` force ou interdit l'envoi distant.
- `--remote <nom>` choisit un serveur distant et active automatiquement le transfert (utilise `config/remotes/<nom>.conf`).

### Exemples :

#### Sauvegarde locale simple

```
./dbbackup backup -u root -p secret -d mydb
```

#### Sauvegarde sans chiffrement

```
./dbbackup backup -u admin -p pass -d production --no-encrypt
```

#### Sauvegarde et envoi vers le remote "prod"

```
./dbbackup backup -u root -p secret -d mydb --remote prod
```

```
dbbackup backup list
```

Liste les sauvegardes locales ou distantes disponibles.

Options :

- `-o|--output <dossier>` analyse un dossier spécifique.
- `--remote` (optionnellement `--remote=<nom>`) affiche les sauvegardes sur un serveur distant. Sans nom, le serveur par défaut est utilisé.

**Exemples :**

Lister les fichiers locaux

```
./dbbackup backup list
```

Lister les sauvegardes distantes depuis "prod"

```
./dbbackup backup list --remote=prod
```

## dbbackup restore

Restaure une base à partir d'une sauvegarde locale ou distante. Le script décompresse, déchiffre et valide la somme de contrôle avant d'utiliser MySQL.

Options :

- `-f|--file <chemin>` fichier de sauvegarde (obligatoire sauf `--list`).
- `-h|--host`, `-P|--port`, `-u|--user`, `-p|--password`, `-d|--database` paramètrent la base cible.
- `--remote` télécharge la sauvegarde depuis le serveur distant actif (utilise le nom passé à `-f` relatif au répertoire distant).
- `--verify` vérifie l'intégrité sans restaurer.
- `--list` affiche les sauvegardes locales disponibles.

Le script vérifie l'existence de la base, peut la créer sur demande, et requiert la confirmation `yes` avant d'écraser une base existante.

### Exemples :

Restaurer une sauvegarde locale chiffrée

```
./dbbackup restore -f backups/mydb_20241110_020000.sql.gz.enc -u root -p secret -d mydb
```

Vérifier uniquement la sauvegarde

```
./dbbackup restore -f backups/mydb_20241110_020000.sql.gz.enc --verify
```

Restaurer un fichier présent sur le serveur distant "prod"

```
./dbbackup restore -f daily/mydb-2024-11-10.sql.gz.enc --remote -u root -p secret -d mydb
```

## dbbackup schedule

Gère les sauvegardes planifiées via cron. Toutes les planifications sont stockées dans config/schedules.conf.

Sous-commandes :

- `schedule add` : ajoute une planification. Nécessite `-n|--name`, `-c|--cron`, `-u|--user`, `-p|--password`, `-d|--database`. `--remote <nom>` rend la planification distante.
- `schedule list` : affiche un tableau des planifications configurées et leur statut.
- `schedule modify <nom>` : met à jour cron, hôte, identifiants ou serveur distant (`--remote` ou `--no-remote`).
- `schedule remove <nom>` : supprime la planification et la tâche cron associée.
- `schedule enable <nom>` / `schedule disable <nom>` : active ou désactive la planification tout en installant/retirant la tâche cron.
- `schedule next` : rappelle la prochaine exécution théorique pour chaque planification active.

### Exemples :

Planifier une sauvegarde quotidienne locale à 02h00

```
./dbbackup schedule add -n daily -c "0 2 * * *" -u root -p secret -d mydb
```

Ajouter une sauvegarde distante

```
./dbbackup schedule add -n daily-remote -c "0 3 * * *" -u root -p secret -d mydb --remote prod
```

Modifier une planification pour retirer le transfert distant

```
./dbbackup schedule modify daily-remote --no-remote
```

Suspendre puis réactiver une planification

```
./dbbackup schedule disable daily
```

```
./dbbackup schedule enable daily
```



## dbbackup remote

Administre les serveurs distants utilisés pour les transferts et restaurations. Chaque serveur correspond à un fichier config/remotes/<nom>.conf.

Sous-commandes :

- remote list : liste les serveurs connus (le serveur par défaut est marqué d'une \*).
- remote add <nom> --host <hôte> --user <utilisateur> [options] : crée un nouveau profil. Options disponibles : --port, --path, --auth key|password, --ssh-key, --password, --verify yes|no, --delete-after yes|no, --set-default.
- remote show <nom> : affiche la configuration détaillée.
- remote remove <nom> : supprime le profil (et réinitialise le défaut si besoin).
- remote set-default <nom> : définit le serveur utilisé par défaut quand --remote n'est pas renseigné.
- remote test <nom> : tente une connexion SSH/SFTP avec les paramètres stockés.

Exemples :

Ajouter un serveur distant basé sur une clé SSH

```
./dbbackup remote add prod --host backup.example.com --user backup --auth key --ssh-key  
~/.ssh/id_rsa --path /srv/backups --set-default
```

Afficher la configuration puis tester la connexion

```
./dbbackup remote show prod  
./dbbackup remote test prod
```

## dbbackup config show

Affiche le contenu de config/dbbackup.conf, le résumé des transferts (config/transfer.conf), la liste des serveurs distants disponibles et les chemins importants (clé de chiffrement, fichier de planification). Utile pour vérifier rapidement la configuration active.

Exemple :

```
./dbbackup config show
```

Le présent document est exclusivement destiné à un usage pédagogique dans le cadre scolaire. Il s'inscrit dans un contexte strictement éducatif et non public.

Toute utilisation, reproduction, diffusion ou exploitation, totale ou partielle, à des fins autres que l'apprentissage notamment à des fins commerciales, professionnelles ou promotionnelles est formellement interdite et pourra faire l'objet de poursuites conformément aux dispositions en vigueur relatives à la propriété intellectuelle. (voir notamment l'article Article L122-5 du Code de la propriété intellectuelle)